ARCHITECTURE DETAILED DESIGN

AlphaOpen Tennis League Progressive Web Application

Control	Value
Document status	Proposed for Architecture Review Board
Version	1.0
Review date	July 26, 2026
System	AlphaOpen Tennis League
Production project	alphaopen-development-2026
Current hosting	https://alphaopen-development-2026.web.app
Architecture posture	Firebase Spark-safe, client-mediated transactions

ARB recommendation: Approve the current Spark-safe architecture as an interim operating model, subject to the controls and remediation plan in Section 15. The design is suitable for a trusted, small league administration model but is not the recommended final state for high-assurance or materially scaled operations.

Decision requested

- Approve the current Firebase Hosting, Authentication, Firestore, and PWA architecture for controlled production use.
- Approve the team-level lineup workflow and the Spark-safe transaction pattern as the current system of record behavior.
- Accept the documented residual risks of privileged client execution, especially exceptional lineup reset validation.
- Approve the future Blaze migration pattern without requiring a Firestore data migration.
- Assign owners and target dates for the conditions of approval.

Document purpose and evidence base

This document describes the as-built application architecture, data model, security model, operational workflows, deployment topology, and planned evolution. It is based on the deployed source configuration and the repository artifacts ALPHAOPEN_APP_SPEC.md, FIRESTORE_DATA_MODEL.md, firebase.json, firestore.rules, firestore.indexes.json, manifest.webmanifest, the route and feature modules, and the Spark-safe lineup workflow implementation.

1. Executive architecture summary

AlphaOpen is a mobile-first, single-page Progressive Web Application for public league information and authenticated league operations. Static HTML, CSS, JavaScript modules, images, and the service worker are served by Firebase Hosting. Google Authentication establishes identity. Cloud Firestore is the operational and public data store. Firestore Security Rules are the primary authorization and workflow enforcement layer.

The application currently operates on the Firebase Spark plan. There is no deployed trusted application server. Standard create, update, approval, publication, and reset actions execute in the browser using Firestore transactions or batched writes. The security design therefore combines role data, immutable snapshots on matchup records, strict document transition rules, and create-only audit/revision records.

Layer	Current design	Assessment
Presentation	Responsive SPA/PWA using HTML, CSS, JavaScript modules and route-based lazy loading	Production
Identity	Google Sign-In through Firebase Authentication; verified-email player linking and admin approval	Production
Application services	Browser modules and Firestore transactions; no deployed Cloud Functions	Interim
Data	Cloud Firestore, default database, nam5 multi-region	Production
Authorization	Firestore Security Rules plus client route visibility	Production
Hosting	Firebase Hosting with SPA rewrite and no-cache code headers	Production
Offline	Service worker app-shell cache; operational writes still require Firestore connectivity	Limited
Delivery	Manual Firebase CLI deployment to one Firebase project	Needs hardening

Architecture quality assessment

Category	Assessment
Strengths	Simple deployment, low operating cost, responsive PWA, direct real-time reads, strong document ownership model, immutable lineup revisions, transaction-based concurrency.
Constraints	No trusted execution tier, no scheduled/background processing, no centralized secrets, limited server-side validation, manual cache versioning, one environment.
Highest residual risk	A privileged user can attempt direct SDK writes. Rules control status transitions, but application-level checks such as score inspection during exceptional reset cannot be independently queried by Firestore Rules.
Recommended disposition	Approve with conditions and define explicit triggers for a Blaze/server migration.

2. Scope, principles, and design decisions

2.1 In-scope capabilities

- Public home, season, match, standings, history, rules, venue, and community content.
- Google authentication, user registration, Player Master identity matching, and Super Admin approval.
- Season, team, roster, venue, matchup, approver, and player administration.
- Captain, EC, Neutral Approver, and Super Admin lineup submission.
- Independent Home/Away team-level approval, rejection, and self-approval tracking.
- Full lineup publication, match scheduling, score entry, calculated points, and standings publication.
- Exceptional reset of both fully approved lineups when no score activity is detected.
- PWA installation and app-shell offline behavior.

2.2 Governing principles

Principle	Application
Canonical identity	Firebase UID identifies the account; Player ID identifies the person across seasons and history.
Season isolation	Operational data is nested below a season and is governed by active season membership.
Least privilege	Guest, Player, Captain, EC, Neutral Approver, and Super Admin access is evaluated separately.
Immutable history	Submitted lineup revisions and workflow actions are create-only.
Atomic state change	Cross-document workflow state is committed in one transaction or batch.
Public/private separation	Public projections are separate from operational and private player records.
Snapshot history	Names, ranks, teams, and actor details are copied into official records to preserve historical meaning.
Plan portability	UI and schema contracts remain stable when protected mutations move to Cloud Functions.

2.3 Key architecture decisions

ID	Decision	Rationale
ADR-001	Progressive Web App	One responsive codebase for phone, tablet, and desktop; avoids native app distribution.
ADR-002	Firebase managed platform	Use Hosting, Authentication, and Firestore to minimize operational overhead.
ADR-003	Spark-safe workflow	Use browser-side Firestore transactions while Blaze is unavailable.
ADR-004	Team-level lineup states	Track Home and Away independently and derive matchup approval status.
ADR-005	Immutable lineup revisions	Never overwrite submitted history; current lineup points to a numbered revision.
ADR-006	Separate reset exception	Fully approved lineups cannot be changed through normal approval; reset always affects both.
ADR-007	On-demand score inspection	Do not maintain scoreActivityStarted; query and re-read line records only when reset is requested.
ADR-008	Future trusted service seam	Expose workflow operations through one client service so Cloud Functions can replace transaction internals.

3. System context

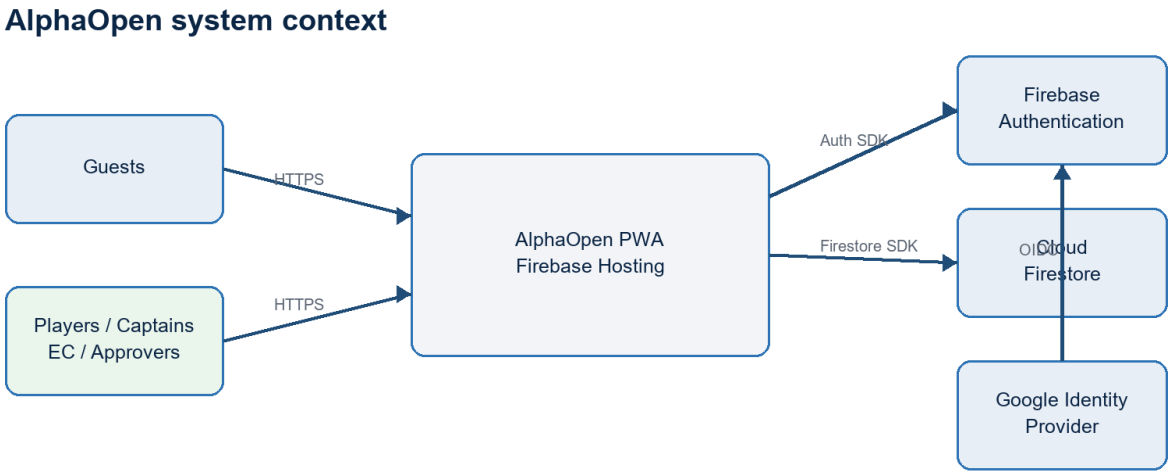

Figure 1. System context and managed-service boundaries

3.1 Actors and trust boundaries

Actor	Trust classification	Primary access
Guest	Untrusted anonymous browser	Published/public documents only
Player	Authenticated active user	Own account, membership, availability, permitted history
Captain	Authenticated operational user	Own team lineup, scheduling, and score responsibilities
EC	Season administrator	Rosters, season structure, venues, disputes, operational visibility
Neutral Approver	Privileged seasonal assignment	Either team submission; independent approval/rejection; reset exception
Super Admin	Highest privilege	Global administration and all operational actions
Firebase services	Managed trusted platform	Identity tokens, rule evaluation, transactions, persistence, hosting

3.2 External dependencies

- Google Identity Provider for interactive account authentication.
- Firebase JavaScript SDK version 12.16.0 loaded from the Google CDN.
- Firebase Hosting, Authentication, and Cloud Firestore in project alphaopen-development-2026.
- Browser platform services: service workers, Cache API, local/session persistence, dialogs, canvas, and clipboard.
- Firebase CLI for rules and hosting deployment.

4. Logical application architecture

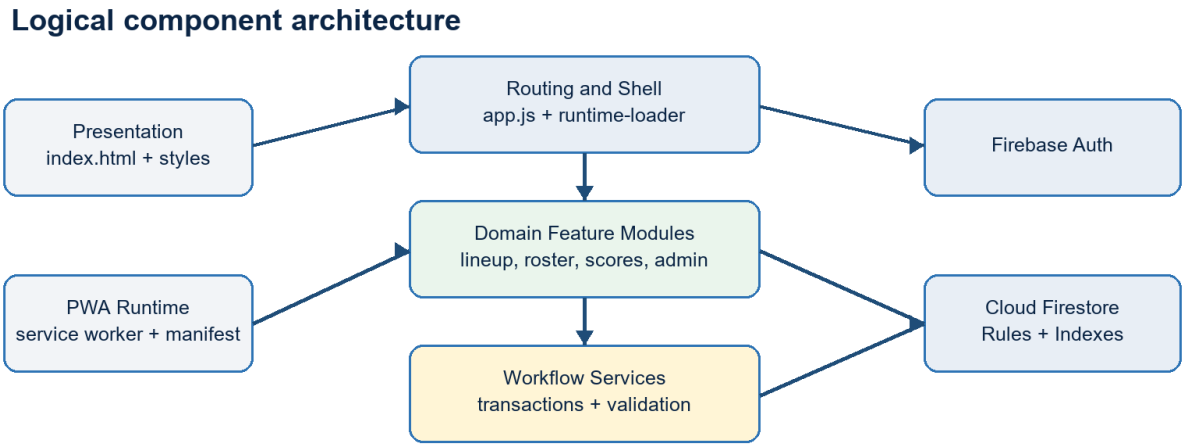

Figure 2. Browser modules and Firebase service boundaries

4.1 Component catalog

Component	Primary modules	Responsibility
Application shell	index.html, styles.css, app.js	Routes, navigation, responsive views, dialogs, common rendering.
Runtime loader	runtime-loader.js and feature bootstraps	Loads core Firebase modules and route-specific modules on demand.
Identity	firebase-auth.js, player-identity.js	Google sign-in, verified email matching, profile provisioning, authorization context.
Data access	firebase-client.js, firebase-data.js	Firebase initialization, public/operational reads, UI data hydration.
Lineup workflow	lineup-submit.js, lineup-approve.js, lineup-reset.js, lineup-workflow-client.js	Draft, submit, approve, reject, publish, and reset transactions.
Match operations	match-management.js, score-rules.js	Schedule entry, score validation, points, completion locking.
Season administration	season-*.js, roster-admin-v3.js, venue-admin.js	Season import, teams, matchups, rosters, venues, reset.
Public projection	public-season-dashboard.js	Publishes guest-safe season data after authorized operational changes.
Offline shell	service-worker.js, manifest.webmanifest, pwa.js	Installability, cache lifecycle, offline navigation fallback.
Security policy	firestore.rules, firestore.indexes.json	Authorization, transition constraints, query support.

4.2 Routing and module lifecycle

The application is a hash-routed SPA. Views are declared as sections in index.html and activated by app.js. The runtime loader imports Firebase core modules and loads route-specific administrative modules only when their route or admin panel becomes active. Separate small bootstrap modules initialize lineup, approval, reset, EC status, and match-management features.

Code assets use explicit query-string versions. Firebase Hosting sets no-cache headers for JavaScript and CSS. The service worker uses a versioned cache, applies network-first behavior to app code, and deletes earlier cache versions during activation.

5. Deployment and runtime topology

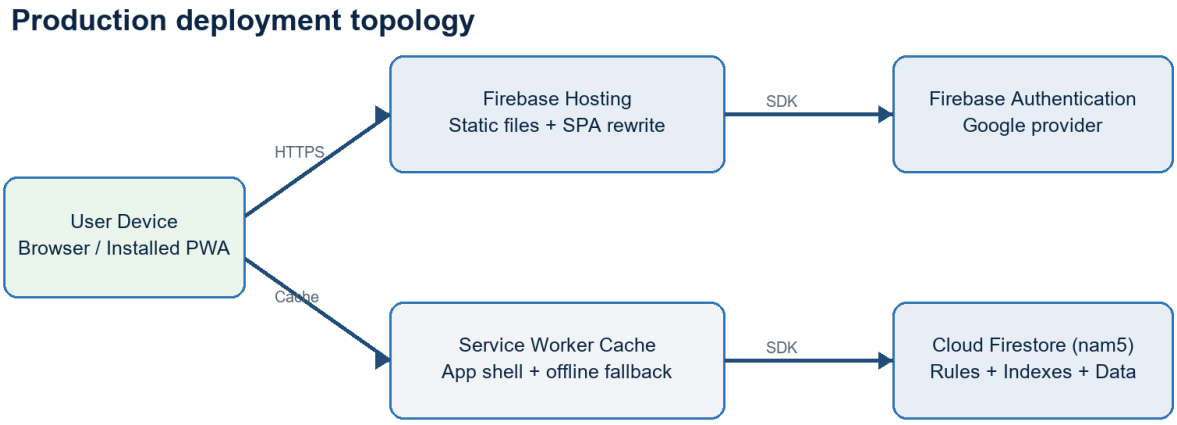

Figure 3. Current Spark-plan production topology

Element	Configuration	Architecture note
Firebase project	alphaopen-development-2026	Single project currently serves production.
Hosting URL	alphaopen-development-2026.web.app	Public endpoint; all paths rewrite to index.html.
Firestore	(default), nam5	Operational, private, and public documents.
Functions	Not configured for deployment	Functions source is retained for future migration but excluded from Spark deployment.
Headers	No-cache for HTML, JS, CSS, manifest	Reduces stale-code risk; service worker provides controlled cache fallback.
Emulators	Auth 9099, Firestore 8080, Hosting 5000	Local validation and security-rule testing.

5.1 Environment and release management

The repository currently targets one Firebase project and contains the Firebase client configuration directly in browser code, which is normal for Firebase clients. Production deployment is performed with the Firebase CLI for Hosting and Firestore Rules. There is no separate staging project, automated promotion workflow, or infrastructure-as-code pipeline.

Condition of approval: Create separate development/test and production Firebase projects, automate rule tests in CI, and require reviewed deployment promotion before operational scale increases.

6. Identity, authorization, and security architecture

6.1 Authentication and registration

1. User signs in with Google through Firebase Authentication.
2. The application requires a verified Google email.
3. The normalized email must match an active Player Master email index.
4. A new user record is created as pending and linked to the canonical Player ID.
5. Super Admin approves the registration and assigns active season membership and roles.
6. The application derives route access from global roles plus current-season member roles.

6.2 Authorization layers

Layer	Mechanism	Security role
Navigation visibility	data-access, data-access-any, data-nav-roles, data-super-admin	User experience only; not a security boundary.
Client guard clauses	Feature modules validate authorization context and team scope	Prevents accidental actions and provides clear errors.
Firestore Rules	Identity, active status, season role, team scope, allowed fields, and transition checks	Primary security boundary.
Transaction invariants	Current revision, current status, operation ID, actor UID, and multi-document state	Concurrency and workflow integrity.
Immutable records	Revision and review documents are create-only	Nonrepudiation and audit preservation.

6.3 Role access matrix

Capability	Guest	Player	Captain	EC	Approver	Super Admin
Public content	Read	Read	Read	Read	Read	Read/write
Own player/account data	-	Read/update limited	Read/update limited	Read	Read	All
Team lineup submit	-	-	Own team	Authorized scope	Either team	Any
Lineup approve/reject	-	-	-	-	Assigned scope	Any
Reset approved lineups	-	-	-	-	Assigned scope	Any
Roster/season operations	-	-	Request/view	Season scope	View	Any
Schedule/score	Published read	Assigned read	Team scope	Season scope	Read	Any
Security administration	-	-	-	-	-	Any

6.4 Security controls and residual risk

Control	Design	Status
Verified identity	Firebase ID token and verified Google email.	Implemented
Private data isolation	Separate playerPrivate and venuePrivate documents.	Implemented
Role enforcement	Rules read user, member, assignment, and matchup snapshots.	Implemented
Privilege snapshot	Matchups carry captain and approver UID lists for efficient rule evaluation.	Implemented
Workflow actor binding	Matchup stores operation ID and actor UID; related writes must match.	Implemented
Immutable workflow history	Submitted revisions and action records cannot be updated or deleted.	Implemented
Exceptional reset score check	Application queries line matches and transaction re-reads them.	Application-enforced
App Check	No enforced App Check attestation.	Gap

Control	Design	Status
Central monitoring	No server event pipeline or alerting.	Gap
Bootstrap Super Admin	Protected email is embedded in client/rules until custom claims are introduced.	Accepted interim risk

7. Data architecture

7.1 Collection hierarchy

Domain	Collections	Purpose
Global identity	users, players, playerPrivate, playerEmailIndex, playerAccountLinks, registrationRequests	Account identity and canonical person records
Global reference	venues, venuePrivate, systemConfig, aoContent	Venues, active season control, public content
Season core	seasons/{seasonId}	Season configuration and status
Season access	members, approverAssignments	Season roles and approval scope
Competition setup	teams, rosterSlots, rosterAssignments, ruleVersions, weeks, matchups	Teams, ranks, rules, schedule structure
Lineup workflow	matchups/{matchupId}/lineups, revisions, lineupReviews	Current lineups, immutable submissions, actions
Match operations	lineMatches, scheduleProposals, scoreSubmissions, scoreDecisions	Line scheduling and scoring
League operations	availability, replacementRequests, latePassRequests, adjustments	Supporting workflows
Results	standings, standingsSnapshots, playoffBrackets	Calculated/public competition results
Audit/notification	auditEvents, users/{uid}/notifications	Operational trace and user messages

7.2 Identifier strategy

Identifier	Use
Firebase UID	Authentication account and role assignment
Player ID	Permanent person identity and historical linkage
Season ID	Partition key for competition operations
Team ID	Season team identity
Assignment ID	Immutable roster assignment interval
Matchup ID	Home-versus-Away team event
Line match ID	Matchup plus line number; normally {matchupId}-L1 through L5
Operation ID	Client-generated idempotency key for lineup action
Revision number	Monotonic lineup submission version within team/matchup

7.3 Public/private partitioning

Public pages must read only explicitly published fields. Operational season documents include member UIDs, internal status, lineup workflow state, and administrative metadata. Personally identifiable information is split into private collections. Official records carry display-name snapshots for historical readability without requiring broad access to private master records.

Data-governance note: Snapshot fields are intentionally denormalized. They preserve historical meaning but require defined correction procedures when an official record is wrong; changing the Player Master must not silently rewrite history.

8. Core schema design

8.1 Identity and master data

Document	Key fields	Purpose
users/{uid}	uid, email, status, profileType, playerId, globalRoles, displayName, lastLoginAt	Account authorization and profile
players/{playerId}	displayName, public fields, active status	Canonical public player identity
playerPrivate/{playerId}	emailNormalized, phone, private notes, status	Restricted PII and identity match
playerAccountLinks/{playerId}	uid, status, approval metadata, link method	One-account-to-player link
seasons/{seasonId}/members/{uid}	roles, teamIds, playerId, status	Season-scoped authorization
teams/{teamId}	name, captainUids, captainPlayerIds, status	Season team master
rosterSlots/{team_rank}	teamId, rankNumber, participation counters	Persistent team-rank slot
rosterAssignments/{assignmentId}	teamId, rankNumber, playerId, type, dates, eligibility, status	Effective-dated player assignment

8.2 Matchup master

Field group	Fields
Identity	matchupId, weekId, stage, homeTeamId, awayTeamId
Authorization snapshots	homeCaptainUids, awayCaptainUids, approverUids
Team states	homeLineupStatus, awayLineupStatus
Overall state	lineupApprovalStatus, bothLineupsSubmitted, lineupsPublished
Revision pointers	homeLineupRevisionNumber, awayLineupRevisionNumber
Actor tracking	homeLineupTracking, awayLineupTracking
Publication	fullyApprovedAt, lineupsPublishedAt
Concurrency/security	lineupWorkflowActorUid, lineupWorkflowOperationId
Reset	approvalCycleNumber, lastLineupReset
Competition	status, completedLineCount, homeTeamPoints, awayTeamPoints, standingsApplied

8.3 Lineup current document and revision

Field group	Definition
Identity	seasonId, matchupId, teamId, revisionNumber, ruleVersionId
State	draft | submitted | approved | rejected
Content	five lines; each has two player IDs/names/ranks and SOR
Validation	passed, errors, checkedAt, checkedBy, ruleVersionId
Submission	submittedByUid/playerId/name/role, submittedAt
Approval	approvedByUid/playerId/name/role, approvedAt
Rejection	rejectionReason, rejectedByUid/playerId/name/role, rejectedAt
Audit	updatedByUid, updatedAt
Revision copy	same payload plus immutable=true in revisions/{revisionNumber}

8.4 Line match

Field group	Definition
Identity	lineMatchId, matchupId, lineNumber, seasonId
Teams/players	homeTeamId, awayTeamId, homePlayers[2], awayPlayers[2], rank snapshots
Lineup linkage	homeLineupRevisionNumber, awayLineupRevisionNumber, lineupState, scoreEntryAllowed

Field group	Definition
Schedule	venueId/name/address, scheduledAt, scheduleStatus
Score	sets, scoreStatus, winnerTeamId, homePoints, awayPoints, completedAt
Audit	createdAt, updatedAt

9. Lineup submission and approval detailed design

9.1 Status model

Team status	Meaning	Next action
pendingSubmission	No sealed submission is available for this team.	Captain/EC/Approver/Super Admin submits
submitted	Current revision is sealed and awaiting a decision.	Approver approves or rejects
approved	Current team revision is approved.	Other team approved or, before full approval, Approver rejects
rejected	Current revision was returned with a reason.	Authorized submitter submits a new revision

Matchup status	Derivation
awaitingSubmission	At least one team remains pending and neither team is rejected.
awaitingApproval	Both teams are submitted/approved but at least one submitted revision needs approval.
rejected	At least one team lineup is rejected.
fullyApproved	Both team lineup statuses are approved.

9.2 Submission transaction

7. Validate request IDs, signed-in actor, role, team relationship, and five-line lineup.
8. Read active roster assignments for the selected team and canonicalize player snapshots and ranks.
9. Read the action, matchup, current lineup, and proposed revision inside a Firestore transaction.
10. Reject replacement of a submitted or approved current lineup.
11. Increment the revision number and write the current lineup plus immutable revision copy.
12. Update the appropriate Home/Away status, revision pointer, tracking map, and derived matchup status.
13. Set workflow actor UID and operation ID on the matchup.
14. Create a write-once submitted action containing prior/new status and result.

9.3 Approval/rejection transaction

15. Only a Neutral Approver assigned to the matchup/season or Super Admin may decide.
16. The Approver may review one submitted team even while the opponent is pending.
17. Approval requires the current status to be submitted; rejection permits submitted or approved before full approval.
18. A rejection requires a reason and clears approval metadata.
19. Self-approval is permitted for the current release and is recorded when submittedByUid equals approvedByUid.
20. The matchup status is recalculated from both team states.
21. When both teams become approved, five line-match records are created or refreshed and score entry is enabled.

22. The decision and all publication writes are committed atomically with a create-only action record.

9.4 Visibility model

Role	Before full approval	After full approval
Captain	Own team lineup	Opponent team status only until publication
EC	Authorized season lineups	Operational status and roster context
Neutral Approver	Both current team lineups	Independent team decisions
Super Admin	All current and historical lineup data	All actions
Player/Guest	Published information only	No sealed drafts or internal decisions

9.5 Requirements variance from original specification

Topic	Earlier baseline	Current design	Disposition
Approver visibility	Original: wait for both submissions	Current: Approver can review the submitted team while the opponent is pending	Approved product decision
Approval granularity	Original: approve both together	Current: approve/reject each team independently; publish only when both approved	Approved product decision
Approver submission	Original: not allowed	Current: Neutral Approver may submit either team and self-approve with audit flag	Approved product decision
Time restrictions	Original: deadlines envisioned	Current: no approval time restrictions	Explicitly deferred
Post-approval change	Original: change/consent workflow	Current: no change request; separate reset-both exception	Approved simplification

10. Reset Approved Lineup exception

10.1 Access and selection

Reset Approved Lineup is a separate screen available only to Neutral Approvers and Super Admins. The user selects Season, Week, and Home Team/matchup. Both approved lineups, submitter/approver metadata, and all five line matches are displayed before reset.

10.2 Eligibility validation

- Matchup must be fullyApproved, or both stored team statuses must be approved.
- Both current lineup documents must exist.
- The application queries every line match and blocks when it finds in-progress/completed states, submitted/confirmed/disputed/published score states, nonzero set scores, nonzero points, a winner, or completedAt.
- The reset transaction re-reads the discovered line documents plus the five canonical line IDs immediately before writing.
- The user must enter a reason, acknowledge the effect, and type the exact Matchup ID.

10.3 Reset transaction result

Record	Result
Home lineup	Delete current document; preserve revisions
Away lineup	Delete current document; preserve revisions
Team statuses	Both pendingSubmission
Matchup status	awaitingSubmission
Publication	lineupsPublished=false; timestamps cleared
Approval cycle	Increment by one
Line matches	Retain schedule documents; set lineupState=awaitingReapproval and scoreEntryAllowed=false
Audit	Create resetBothApprovedLineups action with actor, reason, previous revisions, and cycle

Residual risk: Firestore Rules cannot issue an authorization-time collection query proving that no scored line exists. The application performs the score inspection and transaction re-read. This is reasonable for a rare action restricted to trusted Approvers and Super Admins, but it is the strongest trigger for future trusted-server migration.

11. Scheduling, scoring, and publication

11.1 Match schedule and score

Each fully approved matchup produces five operational line-match documents. Captains may update lines involving their teams; EC and Super Admin users have season-wide management scope. A reset line with scoreEntryAllowed=false or lineupState=awaitingReapproval cannot be scored until both new lineups are fully approved.

11.2 Score validation

- Set scores are structured numeric values rather than free-form result text.
- The scoring module validates completeness and determines the winner.
- Straight-set completion does not permit a third set.
- Split first and second sets require a deciding set.
- Winner receives 14 league points; loser points follow the configured two-set or three-set rules.
- Final score entry requires explicit user confirmation and then disables further normal score update.

11.3 Public projection

Operational writes are followed by publication of guest-safe season dashboard data. The public projection must contain only approved schedules, published lineups, results, standings, and approved venue fields. Private contact details, registration state, availability, sealed lineups, internal notes, and dispute information remain outside the public projection.

11.4 Consistency considerations

Boundary	Mechanism	Implication
Within a workflow	Firestore transaction/batch	Atomic current state, revision, matchup, line records, and action.
Public projection	Follow-on client publication	Eventual consistency; a failed projection may leave public data stale.
Standings	Client/server-derived documents depending workflow	Must be idempotent and protected from duplicate application.
Offline writes	Firestore client behavior	Operational flows should surface connectivity and avoid assuming immediate publication.

12. Transaction, concurrency, and integrity design

Integrity concern	Control
Idempotency	Every official lineup operation carries a random operation ID; duplicate action reads return the prior result.
Optimistic concurrency	Firestore transaction retries when read documents change.
Revision monotonicity	Next revision is max(current lineup revision, matchup pointer) + 1.
Sealed state	Submitted and approved current lineups cannot be overwritten by normal submission.
Cross-document binding	Matchup actor UID and operation ID bind related write permissions in Rules.
State derivation	Matchup approval status is recalculated from Home and Away statuses.
Immutable evidence	Revisions and review actions are create-only.
Reset safety	Known line documents are re-read before reset; score activity causes application failure.

12.1 Rule-evaluation budget

Firestore Rules have expression and document-access limits. Workflow rules use matchup authorization snapshots and a single operation actor binding to avoid repeatedly loading user, member, and assignment records for every write in a multi-document transaction. Emulator tests validate the complete submission, one-sided approval, second-team approval with five line records, unauthorized Captain denial, and reset transaction.

12.2 Failure behavior

Failure	Expected behavior
Validation failure	No official write; display exact lineup or score error.
Authorization failure	Firestore permission-denied; UI restores controls and displays failure.
Concurrent update	Firestore retries transaction; stale state is rejected on re-evaluation.
Partial network failure	Atomic transaction does not partially commit.
Public projection failure	Operational state may commit while public projection remains stale; requires retry/monitoring.
Service worker stale asset	Network-first app code and no-cache hosting headers reduce risk; versioned cache activates on update.

13. Nonfunctional architecture

Quality attribute	Current design	Required control
Availability	Firebase managed services; static hosting remains available independently of custom servers.	Define service objective and status monitoring.
Performance	Static CDN delivery, lazy modules, Firestore direct reads, indexed queries.	Add query budgets and representative device testing.
Scalability	Adequate for a small league; denormalized documents and direct reads scale horizontally.	Review read amplification and public projection strategy before multi-league expansion.
Security	Firebase Auth + Rules + immutable records.	Add App Check, environment separation, custom claims/server controls.
Privacy	Public/private collections and limited contact access.	Document retention, deletion, and incident procedures.
Accessibility	Responsive semantic HTML and mobile layout.	Perform formal WCAG 2.2 AA audit and keyboard/screen-reader testing.
Maintainability	Feature modules and shared workflow service.	Introduce build tooling, linting, typed contracts, and module tests.
Observability	Browser errors and Firebase console.	Add structured event/error monitoring and deployment traceability.
Recoverability	Firestore managed durability; immutable history.	Schedule exports/backups and test restore.
Compatibility	Modern iOS/Android/desktop browsers; PWA manifest.	Maintain supported-browser matrix.

13.1 Data retention and recovery

- Submitted lineup revisions and official workflow actions should be retained for the full league-history period.
- Private player data should follow a documented retention and correction policy.
- Season reset must preserve the season root and immutable audit evidence while deleting explicitly scoped descendants.
- Production Firestore export and restore procedures should be exercised before the next season launch.

14. Testing and architecture assurance

Assurance layer	Mechanism	Coverage
Static application contract	test-prototype.mjs	Routes, views, modules, PWA, encoding, score rules, lineup service usage.
Workflow logic	functions/test/workflow.test.js	Status derivation, five-line validation, score-activity detection.
Rules compilation	Firebase Firestore emulator	Production rules compile without errors.
Role/workflow integration	spark-workflow-rules-test.mjs	Submission, Captain approval denial, one-sided approval, full approval, reset.
Browser smoke test	Local hosted application	Required views and versioned modules load without console errors.
Production verification	Firebase deploy output and HTTPS request	Hosting/rules released; live response is HTTP 200.

14.1 Required pre-release gates

- All JavaScript syntax and static contract tests pass.
- Firestore Rules compile and the Spark workflow emulator test passes.
- Rules changes receive security-focused peer review.
- Representative Captain, Approver, EC, and Super Admin user journeys pass in a non-production Firebase project.
- PWA cache version and module query versions are advanced when deployable assets change.
- Production smoke test confirms route load, authentication, and read/write permission behavior.

15. Risks, conditions, and technical debt

ID	Rating	Risk	Impact	Treatment
R1	High	Privileged workflow logic executes in browser	Direct SDK callers may attempt crafted writes; Rules are the only enforcement tier.	Retain emulator tests; migrate protected mutations to Cloud Functions when trigger is met.
R2	High	Exceptional reset score check is application-enforced	A deliberately modified privileged client could omit a scored line.	Trusted-role restriction now; move validation to server on Blaze.
R3	High	Single Firebase environment	Testing or administrative mistakes can affect production data.	Create separate dev/test and production projects.
R4	Medium	Bootstrap Super Admin email embedded	Long-lived special-case privilege and account concentration.	Provision custom claim or server-managed global role; maintain two-person admin coverage.
R5	Medium	Manual deployment and cache versioning	Human error may release mismatched assets or rules.	CI/CD, manifest generation, deployment approval, rollback procedure.
R6	Medium	Public projection is follow-on client work	Operational and public views may temporarily diverge.	Add retry dashboard now; move projection to event-driven backend later.
R7	Medium	No App Check or centralized telemetry	Abuse and client failures are harder to detect.	Enable App Check and error/event monitoring.
R8	Medium	Specification drift	Earlier specification conflicts with current team-level approval decisions.	Approve this document as the architecture baseline and update product specification.
R9	Low	CDN-loaded SDK and unbundled modules	Dependency or browser-loading behavior is less controlled.	Introduce lockfile/build pipeline and Subresource Integrity where feasible.
R10	Low	No formal WCAG audit	Usability barriers may remain.	Perform WCAG 2.2 AA review before broad launch.

15.1 Conditions of approval

ID	Condition	Owner	Target
C1	Approve and publish the updated product/design baseline reflecting team-level approval and reset-both behavior.	Product owner	Before next season configuration
C2	Create a non-production Firebase project and run role-based end-to-end tests there.	Engineering	Before next material workflow change
C3	Add CI gates for static tests and Firestore Rules emulator tests.	Engineering	Before multi-contributor development
C4	Document backup/export and restore procedures.	Operations	Before production season launch
C5	Define the Blaze migration trigger and budget owner.	ARB/Product	At architecture approval
C6	Review bootstrap admin and App Check strategy.	Security owner	Within one release cycle

16. Future Blaze migration architecture

The application already isolates official lineup mutations behind lineup-workflow-client.js. The Firestore schema, UI payloads, status values, operation IDs, revisions, and audit documents can remain unchanged. A Blaze upgrade replaces the transaction implementation with callable Cloud Functions or HTTPS endpoints using the Firebase Admin SDK.

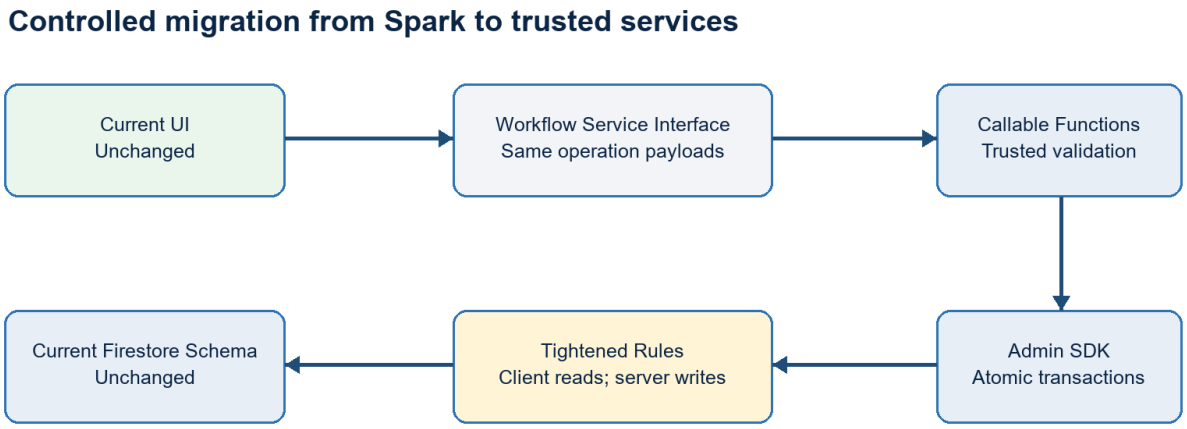

Figure 4. Future trusted mutation path

16.1 Migration steps

23. Enable Blaze billing and required Cloud Functions, Cloud Build, and Artifact Registry APIs.
24. Deploy callable operations for list seasons, submit lineup, decide team lineup, and reset both lineups.
25. Move actor resolution, roster canonicalization, transition validation, score inspection, and publication into trusted code.
26. Switch the workflow service implementation from Firestore transactions to callable requests.
27. Tighten Rules so official workflow documents are read-only to browser clients; retain draft access where appropriate.
28. Enable Firebase App Check enforcement after monitoring legitimate traffic.
29. Add event-driven public projection, notification, and audit functions as separate idempotent handlers.
30. Run dual-path test fixtures and cut over without data migration.

16.2 Recommended migration triggers

Trigger class	Threshold
Security	ARB requires server-side proof of no score activity or stronger nonrepudiation.
Scale	Multiple leagues/seasons, materially more users, or increasing direct-client abuse risk.
Automation	Need scheduled deadlines, notifications, penalties, projections, or background reconciliation.

Trigger class	Threshold
Operations	Need centralized audit events, alerting, retry queues, or service-level objectives.
Integration	Need email/SMS, payments, court systems, or third-party APIs requiring secrets.

17. Open architecture decisions for ARB

ID	Question	Required decision
A1	Is the Spark-safe privileged-client model acceptable through the next season?	Approve interim / require Blaze before launch
A2	Is self-approval by a Neutral Approver acceptable when explicitly recorded?	Accept / require separate approver
A3	Should a Neutral Approver be season-wide or restricted to matchup assignments?	Season / week / matchup policy
A4	What event triggers mandatory Blaze migration?	Date, user volume, security control, or feature dependency
A5	What are retention periods for private player data and immutable competition records?	Retention schedule
A6	What RTO/RPO applies to season operations?	Recovery objectives
A7	Who owns production deployment approval and emergency rollback?	Named operational owner
A8	Which earlier specification clauses are formally superseded by this document?	Baseline reconciliation

17.1 Proposed ARB disposition

Proposed decision: Approve with conditions C1-C6. Permit Spark-safe operation for the current trusted league-administration scope. Require a Blaze migration before introducing untrusted delegated administrators, secret-bearing integrations, automated deadline enforcement, or a security requirement for server-side reset validation.

Appendix A. Lineup state transition reference

From	To	Actor	Guard/effect
pendingSubmission	submitted	Captain, EC, Approver, Super Admin	Valid five-line roster; new revision and action
rejected	submitted	Captain, EC, Approver, Super Admin	Corrected valid lineup; new revision
submitted	approved	Approver, Super Admin	Current revision; decision audit
submitted	rejected	Approver, Super Admin	Required reason
approved	rejected	Approver, Super Admin	Only before matchup is fullyApproved
approved + approved	fullyApproved	System transaction	Publish both and enable five line matches
fullyApproved	awaitingSubmission	Approver, Super Admin	Reset both, no detected score activity, reason and exact confirmation

Appendix B. Firestore Rules ownership summary

Path	Ownership rule
users	Owner limited profile updates; Super Admin account management
players / playerPrivate	Public active player read vs restricted PII
seasons / members	Active member reads; Super Admin writes
teams / rosters / weeks	Season members read; EC/Super Admin write
matchups	Workflow fields through validated role/status transitions; admin structural writes
lineups	Team-scoped draft/submission; reviewer decisions through operation binding
revisions / lineupReviews	Create-only official evidence
lineMatches	Reviewer publication/reset fields; Captain/EC schedule and score fields
approverAssignments	Authorized self/EC reads; Super Admin writes
standings / snapshots	Member reads; protected calculated writes
auditEvents	EC reads; protected writes
unmatched paths	Deny by default

Appendix C. Architecture evidence

Artifact	Evidence
ALPHAOPEN_APP_SPEC.md	Product scope, roles, nonfunctional requirements, original workflow baseline
FIRESTORE_DATA_MODEL.md	Canonical schema, identifiers, collections, ownership, indexes
firebase.json	Hosting, Firestore, emulator, and deployment configuration
firestore.rules	Authorization and workflow transition enforcement
firestore.indexes.json	Composite query support
firebase-auth.js	Authentication, registration, profile, and role derivation
runtime-loader.js / service-worker.js	Module lifecycle, caching, and PWA runtime
lineup-workflow-client.js	Spark-safe official workflow transactions
lineup-submit.js / lineup-approve.js / lineup-reset.js	Role-specific UI workflows
match-management.js / score-rules.js	Scheduling, scoring, locking, and points
test-prototype.mjs / emulator tests	Architecture and security assurance evidence

Appendix D. Glossary

Term	Definition
ARB	Architecture Review Board
EC	Executive Committee
PWA	Progressive Web Application
SOR	Sum of roster ranks for the two players on a doubles line
Spark	Firebase no-cost plan used by the current deployment
Blaze	Firebase pay-as-you-go plan required for the planned Cloud Functions tier
Current lineup	Mutable pointer/document representing the latest team revision and state
Revision	Immutable copy of a submitted lineup
Line match	One of five doubles matches within a team matchup
Public projection	Guest-safe derived document set published from operational data